Project Development Phase

Project Title	implement client script and ui policy (imcident)
Team Name / ID	s.ezhilmanibala,ilakkiya, durai Balaji,Saravanan, lathika
Team Members	team3
Date	30.09.2026

1. Development Approach

- **Methodology:** Iterative. Each task (UI Policy, action, script) was built, saved and tested on the Incident form before starting the next, so errors were caught early.
- **Version control:** no GitHub repository is used; ServiceNow keeps record version history and changes are tracked in the instance, with script copies kept in Google Drive.

2. Module-wise Implementation

Module	Implementation summary	Developer	Status
UI Policy: High Impact Control	Condition Impact is 1 - High, Reverse if false = true; action makes Assignment group mandatory (no code)	[Name]	Done
UI Policy Action: Urgency	Sets Urgency read-only while the policy condition is true (no code)	[Name]	Done
onChange Client Script	JavaScript; uses g_form.setValue and g_form.addInfoMessage to set Urgency = 1	[Name]	Done
onSubmit Client Script	JavaScript; checks impact and assigned_to, shows g_form.showErrorBox and returns false	[Name]	Done
onCellEdit Client Script	JavaScript; alert() and callback(false) to cancel State edit in list	[Name]	Done

3. Key Code Snippets

onChange script - when Impact changes to High, it sets Urgency to High and informs the user (it ignores the initial form load):

onSubmit script - blocks the save for High-impact incidents when Assigned To is empty; returning false stops the submit:

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') { g_form.showErrorBox('assigned_to', 'Assigned To is
        mandatory for High impact incidents.'); return false;
    }
    return true;
}
```

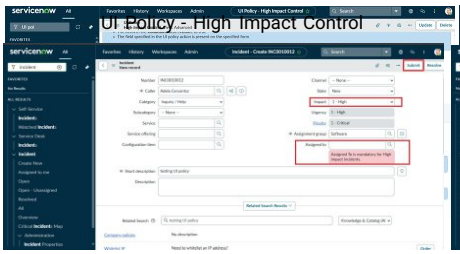
onCellEdit script - cancels any State edit made directly in list view:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.'); callback(false);
}
```

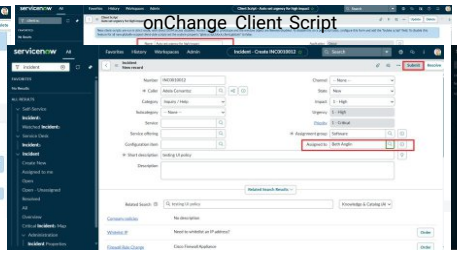
4. Integration

- Frontend, backend and database are all part of the ServiceNow platform, so no external API or connection is needed. The UI Policy and Client Scripts are attached to the Incident table; they run in the browser through the g_form API, and valid records are then saved by the platform.
- **Issues faced:** the UI Policy and onChange script both act on Urgency / Impact at the same time. It was solved by keeping the UI Policy for read-only / mandatory behaviour and the script only for setting the value.

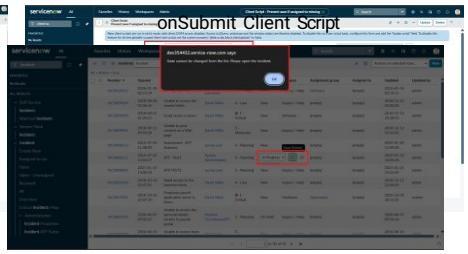
5. Screenshots of Output



Error when Assigned To is empty



Successful save with Assigned To



Alert on State edit in list

6. Challenges & Solutions

- **Challenge:** scripts did not run as expected at first - **Solution:** checked the Impact choice value ('1') and the script Type / Field name, then tested again.
- **Challenge:** the same rules must not affect other impact levels - **Solution:** used the condition Impact is 1 - High with Reverse if false, and tested by changing Impact to Medium.
- **Challenge:** users could still change State from the list - **Solution:** added an onCellEdit script for the State field.

7. Coding Standards & Practices

- Descriptive record names (for example 'Prevent save if Assigned To missing'), lower-case field names as in the dictionary, short comments and one script per purpose.
- No folder structure or commits are used in ServiceNow; instead every script is peer-reviewed by another member before it is marked done, and only the standard g_form API is used (no DOM access).

8. Database & API Implementation

No new tables were created; the existing Incident table is used. The client-side APIs used are:

API / method	Used in	Purpose
g_form.getValue('field')	onSubmit	Read Impact and Assigned To values
g_form.setValue('urgency', '1')	onChange	Set Urgency to High
g_form.addInfoMessage(text)	onChange	Show an information message
g_form.showErrorBox(field, text)	onSubmit	Show an error under the field
alert(text) and callback(false)	onCellEdit	Warn the user and cancel the list edit

9. Progress Report

Week	Work completed	Next steps
Week 1	Requirement analysis and design of the rules	Create the UI Policy and actions
Week 2	UI Policy High Impact Control with Assignment group and Urgency actions	Create Client Scripts
Week 3	onChange, onSubmit and onCellEdit scripts created	Run all test cases
Week 4	Testing (7 of 7 passed) and documentation	Final report and submission

10. Notes / Space for Diagrams

